

VIVA DEFENSE MASTER PLAYBOOK & ORAL DEFENSE DOSSIER

Airtight Theoretical Proofs, Word-for-Word Recital Answers & Emergency Survival Protocols

SYSTEM: ARFOM-DB (12 Tables, 4 Triggers, 5 B-Tree Indexes) | TARGET: HOSTILE VICE DEAN EVALUATION

PART 1: THE 5 MANDATORY BUSINESS RULES (THE TEACHER'S RUBRIC)

#	Mandatory Business Rule	Relational Mechanism	File & Line	10-Second Viva Speech
1	One seat per passenger	UNIQUE(flight_id, seat_id)	tickets table	Storage-engine B-Tree uniqueness barrier. Rejects any duplicate seat allocation on the same flight leg.
2	Aircraft-capacity limit	trg_verify_aircraft_capacity	BEFORE INSERT	Trigger counts uncanceled tickets vs total_capacity. Throws Capacity Overrun exception on overrun.
3	Non-negative baggage	CHECK(weight_kg >= 0.00)	baggage table	Declarative domain check constraint. Rejects negative scale weight or fees at SQL parse level.
4	Check-in eligibility	trg_verify_checkin_eligibility	BEFORE INSERT	Verifies parent booking is CONFIRMED and ticket is ISSUED. Automatically sets ticket to CHECKED_IN.
5	Strict refund bounds	trg_enforce_refund_bounds	BEFORE INSERT	Enforces refund <= fare_amount. Blocks excessive refunds as accounting anomaly and sets ticket to CANCELLED.

PART 2: DBMS CRASH COURSE (ELI5 - ZERO-BACKGROUND MASTERY)

1. Primary Key vs. Foreign Key

A Primary Key uniquely identifies a row in a table. It cannot be NULL and cannot have duplicates. A Foreign Key is a column in a child table pointing to a PK in a parent table to stop orphan records.

Example: airport_code ('DEL') in airports is PK. In tickets, booking_id is an FK pointing to bookings(booking_id).

2. Superkey vs. Candidate Key

A Superkey is any set of columns that uniquely identifies a row (even if it has redundant extra columns). A Candidate Key is a minimal superkey with zero redundant columns. The Primary Key is simply whichever candidate key you choose as primary.

Example: In passengers, (passport_number, email) is a superkey, but passport_number alone is a candidate key.

3. Normal Forms: 1NF -> 2NF -> 3NF -> BCNF

1NF: Atomic cells (no comma-separated lists/arrays). 2NF: In 1NF + No Partial Key Dependencies (every non-key attribute depends on the full composite PK). 3NF: In 2NF + No Transitive Dependencies (A -> B -> C). BCNF: Every determinant is a superkey.

Example: Splitting baggage items into a separate table reaches 1NF. Splitting routes from flights eliminates transitive chains and reaches 3NF.

4. Lossless Join & Dependency Preservation

Lossless Join means joining decomposed tables via natural join reconstructs the exact original relation with zero fake/spurious rows. Dependency Preservation means all functional dependencies can be validated locally in individual tables without expensive cross-table joins.

Example: Common join column between related tables is always a Candidate Key (e.g., airport_code in routes, aircraft_id in seats).

PART 3: 16 HOSTILE VIVA QUESTIONS & EXACT WORD-FOR-WORD RECITALS

Q1: Why did you normalize to 3NF instead of BCNF? Did you fail to reach BCNF?

Evaluator Intent: Does this student understand the theoretical difference between 3NF and BCNF, or did they blindly follow a syllabus prompt?

RECITE OUT LOUD (15 SECONDS):

Our schema strictly satisfies Boyce-Codd Normal Form (BCNF) across all 12 tables, because in every single table, every determinant in our functional dependency set is a superkey. We framed our presentation under 3NF strictly to align with the syllabus rubric ('up to 3NF'). In classical database theory (Bernstein's Synthesis), 3NF guarantees both Lossless Join and 100% Dependency Preservation. In our specific domain, we achieved full BCNF compliance while preserving 100% of functional dependencies locally without requiring expensive inter-table assertions.

PLAIN-ENGLISH CONCEPT (ELI5):

In 3NF, $X \rightarrow Y$ allows Y to be a prime attribute even if X is not a superkey. In BCNF, X must ALWAYS be a superkey. In our tables, every single determinant X is already a Primary Key, so we hit BCNF automatically!

HOSTILE TRAP WARNING: NEVER say 'BCNF was too hard' or 'We didn't need BCNF.' Say 'Our schema strictly satisfies BCNF.'

Q2: Why do you have BOTH a UNIQUE constraint on (flight_id, seat_id) AND an aircraft capacity trigger? Isn't that redundant?

Evaluator Intent: Can you distinguish between micro-level relational uniqueness and macro-level business capacity invariants?

RECITE OUT LOUD (15 SECONDS):

That is layered defensive database engineering addressing two fundamentally distinct invariants. $\text{UNIQUE}(\text{flight_id}, \text{seat_id})$ enforces Micro-Invariant Uniqueness -- preventing two passengers from being allocated the same physical seat on a flight leg. The trigger `trg_verify_aircraft_capacity` enforces Macro-Invariant Airframe Capacity. For example, if operational dispatch dynamically swaps an aircraft on a flight from an Airbus A321 (220 seats) down to an A320 (180 seats), previously booked seat IDs might all be unique, but would violate physical airframe capacity. The trigger acts as an active barrier preventing overbooking beyond certified equipment limits.

PLAIN-ENGLISH CONCEPT (ELI5):

Unique seat numbers stop two passengers sitting on seat 5A. The capacity trigger stops the airline selling 200 tickets on a plane that only has 180 seats.

HOSTILE TRAP WARNING: Do NOT say 'I added both just in case.' Use the exact terms Micro-Level Uniqueness and Macro-Level Airframe Capacity.

Q3: Why use surrogate integer keys (booking_id, ticket_id) instead of natural keys (booking_ref, passport_number)?

Evaluator Intent: Do you understand B-Tree index storage, leaf page fragmentation, and RAM buffer cache efficiency?

RECITE OUT LOUD (15 SECONDS):

We used natural keys where domain standards are globally immutable -- specifically CHAR(3) for IATA airport codes. However, for high-write transactional tables like bookings, tickets, and payments, we deliberately used monotonic integer surrogate keys for three enterprise reasons: 1) B-Tree Page Fill Factors: Sequential integers append cleanly to the right of B-Tree pages, eliminating index page fragmentation caused by random strings. 2) Memory Efficiency: Joining 4-byte integers across 6 downstream tables consumes significantly less buffer cache RAM than joining 20-character strings. 3) Business Logic Isolation: If a customer updates their PNR or email, zero foreign keys break across downstream operational ledgers.

PLAIN-ENGLISH CONCEPT (ELI5):

Computers compare integers in 1 CPU cycle. Comparing long strings takes many cycles and causes page splits on disk B-Trees.

HOSTILE TRAP WARNING: Do NOT say 'Auto-increment is just easier.' Emphasize B-Tree leaf page fragmentation and buffer cache efficiency.

Q4: Why is aircraft_id in seats ON DELETE CASCADE, but in flights it is ON DELETE RESTRICT?

Evaluator Intent: Do you understand referential integrity actions, weak composition entities, and commercial data safety?

RECITE OUT LOUD (15 SECONDS):

Because seats represents an existence-dependent physical component of an airframe (a weak composition entity). A physical seat cannot exist without the plane it is bolted into; when a retired plane is scrapped, its physical seat layout cascades cleanly. In contrast, flights represents an active commercial commitment. If an aircraft has active scheduled flights with booked passengers, deleting the aircraft must be strictly BLOCKED (RESTRICT). This creates a defensive safety net: an in-service aircraft can never be deleted while assigned to flights, while decommissioned aircraft can safely purge their seat maps.

PLAIN-ENGLISH CONCEPT (ELI5):

Scrapping a plane scraps its chairs (CASCADE). But you cannot delete a plane if it has passengers scheduled to fly on it tomorrow (RESTRICT).

HOSTILE TRAP WARNING: Point out that flights(aircraft_id) RESTRICT protects in-service planes and booked tickets from accidental deletion!

Q5: Why is fare_amount stored inside tickets when you could calculate it from a fares table?

Evaluator Intent: Do you understand point-in-time financial auditability and GAAP compliance vs naive over-normalization?

RECITE OUT LOUD (15 SECONDS):

Storing fare_amount directly on the tickets table is a deliberate architectural decision required for Temporal Immutability and Financial Accounting Standards (GAAP). Airline fares fluctuate dynamically due to yield management and demand algorithms. If we calculated ticket prices dynamically via a join to a live fare matrix, every time the airline adjusted prices, the calculated historical revenue of flights flown three months ago would retroactively change, corrupting financial ledgers. Storing the transacted fare at the instant of issuance maintains immutable point-in-time audit integrity.

PLAIN-ENGLISH CONCEPT (ELI5):

If you buy milk for Rs. 50, and next week the store raises milk to Rs. 60, your old printed receipt shouldn't magically change to Rs. 60.

HOSTILE TRAP WARNING: Never apologize for storing fare_amount. Proudly call it 'Temporal point-in-time auditing under GAAP compliance.'

Q6: What happens under heavy concurrent booking traffic? Won't two users booking the same seat cause a race condition in your trigger?

Evaluator Intent: Transaction isolation, row-level locking, and ACID storage engine guarantees under high concurrency.

RECITE OUT LOUD (15 SECONDS):

In an enterprise production environment, concurrency is resolved through two defensive tiers: 1) Transaction Isolation & Row-Level Locking: Booking transactions execute under ISOLATION LEVEL REPEATABLE READ or acquire pessimistic row locks via `SELECT aircraft_id FROM flights WHERE flight_id = ? FOR UPDATE`. This serializes concurrent booking requests against that flight leg. 2) Atomic Storage Engine Barrier: Even if two concurrent threads bypassed the application trigger simultaneously, the database's declarative constraint `UNIQUE (flight_id, seat_id)` is enforced atomically at the storage engine's B-Tree leaf level. The B-Tree page latch guarantees that one thread commits and the second thread is immediately rejected with a duplicate key exception.

PLAIN-ENGLISH CONCEPT (ELI5):

Even if two users click 'Buy' at the exact same millisecond, the database index physically only allows one row to be written. The second gets an instant error.

HOSTILE TRAP WARNING: Do NOT say 'Concurrency is handled by the front-end.' Concurrency in DBMS is handled by transaction isolation levels and B-Tree unique latches.

Q7: Why did you write triggers instead of declarative CHECK constraints for capacity and check-in eligibility?

Evaluator Intent: SQL standard declarative constraint boundaries vs procedural extensions.

RECITE OUT LOUD (15 SECONDS):

Because ANSI SQL standards strictly limit declarative CHECK constraints to single-row, single-table attribute evaluations. A CHECK constraint cannot execute subqueries or inspect parent table states across foreign tables -- for instance, checking bookings.booking_status from inside a checkins insert, or calculating COUNT(tickets) against aircraft.total_capacity. In relational database engines like PostgreSQL and MySQL, cross-table invariant assertions must be enforced via Procedural Triggers (BEFORE INSERT), which execute with indexed O(1) lookups.

PLAIN-ENGLISH CONCEPT (ELI5):

CHECK constraints only know what is in the row you are currently typing. They cannot look into other tables. Triggers can query any table.

HOSTILE TRAP WARNING: Do NOT say 'I like triggers better.' Say 'Declarative CHECK constraints cannot execute subqueries across foreign tables in the SQL standard.'

Q8: Why is cancellations a separate table with ticket_id UNIQUE instead of just setting ticket_status = 'CANCELLED' in tickets?

Evaluator Intent: Sparse tables, null anomalies, and 3NF normalization vs audit ledgers.

RECITE OUT LOUD (15 SECONDS):

That directly satisfies Third Normal Form (3NF) and Financial Audit Separation: 1) Elimination of Null Anomalies: Cancellation metadata (cancellation_timestamp, refund_amount) applies only to cancelled tickets. Adding these columns directly into tickets would force 95% of active issued tickets to store NULL values, resulting in sparse, inefficient tables. 2) Audit Ledger Integrity: A cancellation is a distinct financial accounting event involving refund capital disbursement. A dedicated ledger with ticket_id UNIQUE preserves an unalterable audit trail and links cleanly to payment reconciliation.

PLAIN-ENGLISH CONCEPT (ELI5):

You don't put a 'Date of Death' column on a live employee badge table; you keep termination records in a dedicated audit log.

HOSTILE TRAP WARNING: Mention that it avoids 'Sparse Tables and Null Anomalies in dense relational pages.'

Q9: What index structures did you create and why?

Evaluator Intent: Indexing strategy, B-Tree selectivity, and query optimization paths.

RECITE OUT LOUD (15 SECONDS):

We created secondary B-Tree composite indexes targeted at high-frequency query access paths: 1) idx_flights_search on flights(route_id, scheduled_departure, flight_status): Optimizes customer flight searches using an index-only range scan without full-table scans. 2) idx_bookings_pnr on bookings(booking_ref): Guarantees O(log N) hash-like lookup speed when ground staff query a 6-character PNR. 3) idx_tickets_booking on tickets(booking_id) and idx_tickets_flight on tickets(flight_id): Accelerates multi-table joins when assembling real-time flight manifests and calculating occupancy yield.

PLAIN-ENGLISH CONCEPT (ELI5):

An index is like the index at the back of a textbook. Instead of reading all 500 pages to find 'Delhi flight', you jump directly to page 42.

HOSTILE TRAP WARNING: Always specify that they are 'Secondary B-Tree Composite Indexes' with high selectivity.

Q10: Prove that your schema decomposition has no insertion, deletion, or update anomalies.

Evaluator Intent: Practical proof of 3NF normalization benefits across the data lifecycle.

RECITE OUT LOUD (15 SECONDS):

We can prove this by inspecting our decoupled entities: 1) No Insertion Anomaly: We can add a newly commissioned airport into airports or a new plane into aircraft without requiring any flights, bookings, or passengers to exist. In the unnormalized relation (UNF), an airport could not exist without a booking reference. 2) No Deletion Anomaly: If a passenger cancels their booking, or if old booking records are archived, the physical flight corridor, schedule, and aircraft definitions remain intact. 3) No Update Anomaly: If Indira Gandhi International Airport changes its name, we update a single row in airports. All downstream routes and flights instantly reflect the update via foreign key reference without inconsistent duplicates.

PLAIN-ENGLISH CONCEPT (ELI5):

Decoupling independent entities means updating an airport name in 1 place updates it everywhere without contradictory data.

HOSTILE TRAP WARNING: State all 3 anomalies clearly: Insertion, Deletion, and Update.

Q11: What is a Weak Entity in your schema, and how is it identified?

Evaluator Intent: Relational entity modeling semantics, existence dependency, and identifying composite keys.

RECITE OUT LOUD (15 SECONDS):

In our schema, seats and baggage function as Weak Entities. A seat cannot exist without an aircraft (composite key aircraft_id + seat_number). A bag cannot exist without an originating check-in event (checkin_id). Both cascade on parent deletion (ON DELETE CASCADE).

PLAIN-ENGLISH CONCEPT (ELI5):

A weak entity cannot be uniquely identified by its own attributes alone; it borrows part of its identity from a parent owner.

HOSTILE TRAP WARNING: Quote the composite unique constraint: UNIQUE (aircraft_id, seat_number).

Q12: How do your triggers maintain state machine transitions automatically?

Evaluator Intent: Active database automation, automated side effects, and lifecycle integrity.

RECITE OUT LOUD (15 SECONDS):

Our triggers execute active state machine transitions: Inserting into checkins triggers an update setting tickets.ticket_status = 'CHECKED_IN'. Inserting into cancellations triggers an update setting tickets.ticket_status = 'CANCELLED'. This guarantees that application software cannot accidentally leave tickets in an inconsistent or out-of-sync state.

PLAIN-ENGLISH CONCEPT (ELI5):

The database acts as an active gatekeeper that guarantees lifecycle states remain valid without relying solely on client code.

HOSTILE TRAP WARNING: Call it 'Engine-enforced state machine automation.'

Q13: Explain Lossless Join Decomposition and how your schema satisfies it.

Evaluator Intent: Formal mathematical relational theorem: $R1 \cap R2 \rightarrow R1 \cup R2$ in F^+ .

RECITE OUT LOUD (15 SECONDS):

A decomposition of relation R into R1 and R2 is Lossless if and only if $R1 \cap R2 \rightarrow R1 \cup R2$ in F^+ . In our schema, every single join condition between related tables is a Primary Key: airports INTERSECT routes = airport_code (PK of airports); aircraft INTERSECT seats = aircraft_id (PK of aircraft); bookings INTERSECT tickets = booking_id (PK of bookings). Because the common attribute is always a candidate key, joining tables reconstructs the universal relation with zero spurious or false tuples.

PLAIN-ENGLISH CONCEPT (ELI5):

Lossless join means when you stitch the tables back together with JOIN, you get the exact original data back -- no ghost rows.

HOSTILE TRAP WARNING: Quote the formula: $R1 \cap R2 \rightarrow R1 \cup R2$.

Q14: Explain Dependency Preservation and why your schema preserves 100% of FDs.

Evaluator Intent: Bernstein's synthesis theorem: $(F1 \cup F2 \cup \dots \cup Fk)^+ == F^+$.

RECITE OUT LOUD (15 SECONDS):

Dependency Preservation means the union of all functional dependencies projected onto our individual tables equals the global closure: $(F1 \cup F2 \cup \dots \cup Fk)^+ == F^+$. In our schema, all 18 minimal functional dependencies are enforced locally inside their single projected table via Primary Keys and UNIQUE constraints without requiring expensive cross-table joins during every insert or update.

PLAIN-ENGLISH CONCEPT (ELI5):

Dependency preservation means you don't need to join 3 tables just to check if an insert is legal.

HOSTILE TRAP WARNING: State that all 18 functional dependencies are enforced locally via PK/UQ.

Q15: What is the difference between Logical and Physical Data Independence in your project?

Evaluator Intent: ANSI/SPARC 3-schema architecture in practical database deployment.

RECITE OUT LOUD (15 SECONDS):

1) Logical Data Independence is demonstrated through our Operational Views (`view_flight_manifest`, `view_flight_occupancy`). If internal tables change or columns are added, the conceptual views consumed by ground applications remain constant. 2) Physical Data Independence is demonstrated through our Secondary B-Tree Indexes; we can add, drop, or rebuild indexes like `idx_flights_search` without altering a single application SQL query or changing table schemas.

PLAIN-ENGLISH CONCEPT (ELI5):

Logical independence protects user queries from schema changes. Physical independence protects queries from storage/indexing changes.

HOSTILE TRAP WARNING: Use `view_flight_manifest` as the logical example and `idx_flights_search` as the physical example.

Q16: How does your database handle refund deductions? Why not just refund 100%?

Evaluator Intent: Financial domain rules, business check constraints, and revenue leakage prevention.

RECITE OUT LOUD (15 SECONDS):

Airlines deduct cancellation penalties based on booking cancellation windows. In our seed data, Ticket #5 had a fare of Rs. 8,500.00 and was refunded Rs. 6,800.00, retaining Rs. 1,700.00 as penalty profit. Trigger `trg_enforce_refund_bounds` guarantees refunds cannot exceed base fare. Query #5 tracks net retained cancellation penalties across the system.

PLAIN-ENGLISH CONCEPT (ELI5):

Cancellation fee = Paid Fare minus Refund Amount. The trigger guarantees $\text{refund} \leq \text{fare}$ so the airline never loses money.

HOSTILE TRAP WARNING: Refer to Query #5 in `sql/03_queries_and_views.sql`.

PART 4: EMERGENCY SURVIVAL PROTOCOL (THE POKER FACE)

- 1. If She Interrupts Mid-Sentence**

Do not look defensive or flustered. Stop speaking instantly, nod once, let her finish, then respond calmly:

Example: 'Yes, Vice Dean, that directly relates to our referential constraint in table...'
- 2. If She Asks a Term You Don't Know (The Pivot Script)**

Do not guess or use fake buzzwords. Throw the ball back into her court calmly:

Example: 'In our operational architecture, we enforced that directly at the storage engine layer via triggers. Could you specify which table's lifecycle you would like to see that applied to?'
- 3. The 3-Second Rule (Executive Composure)**

When she finishes asking a hostile question, count 1... 2... 3... in your head before answering.

Example: A calm 3-second pause makes you look like an architect analyzing tradeoffs, not a panicked student reciting memorized notes.

PART 5: 12-TABLE RAPID MENTAL MAP (ONE-LINE MEMORY TRIGGERS)

airports	Physical terminals (DEL, BOM). Primary Key = airport_code CHAR(3).
routes	Flight corridors between 2 airports with distance_km. Enforces origin != dest.
aircraft	Physical airframes with certified total_capacity and model code.
seats	Physical chairs bolted inside airframe. Cascades on aircraft delete.
flights	Scheduled trips on a route using an aircraft with departure and arrival times.
passengers	Customer identities with unique passport numbers and email addresses.
bookings	Commercial order tokens containing 6-character alphanumeric PNR.
tickets	Central junction table connecting booking, flight, and seat. UNIQUE(flight, seat).
payments	Monetary audit record (UPI/Card). Enforces positive transacted amount.
checkins	1:1 gate clearance creating barcode boarding pass for confirmed tickets.
baggage	Checked luggage custody tracking with non-negative weight constraint.
cancellations	1:1 revocation ledger enforcing refund_amount <= ticket fare paid.

PART 6: INSTANT DEMO VERIFICATION COMMANDS (LIVE EXECUTION)

- Run Full Automated Test Suite (6 Negative Tests + Seed Verification)**

Execute in terminal to prove 100% database constraint and trigger enforcement in 3 seconds:

Example: python3 demo_runner.py --demo
- Open Vector Visual Dashboards in Browser (ER Diagram, Data Dictionary, 3NF Proofs)**

Opens the complete high-contrast visual portal for Review 2 visual presentation:

Example: open visuals/index.html